

```

import pandas as pd
import numpy as np

# Load the dataset from CSV
df = pd.read_csv('/content/enjoysport.csv')

# Separate concepts and target
concepts = df.iloc[:, :-1].values
target = df.iloc[:, -1].values

def find_s_algorithm(concepts, target):
    # Initialize the most specific hypothesis (h) with the first positive example
    # If the first example is negative, we need to find the first positive one.
    h = None
    for i, concept in enumerate(concepts):
        if target[i].lower() == 'yes': # Corrected to handle lowercase 'yes'
            h = list(concept) # Convert to list to make it mutable
            break

    if h is None:
        return "No positive examples found, cannot find a specific hypothesis."

    # Iterate through the remaining training examples
    for i, concept in enumerate(concepts):
        if target[i].lower() == 'yes': # Corrected to handle lowercase 'yes'
            for j in range(len(h)): # Compare attributes
                if h[j] != concept[j]:
                    h[j] = '?' # Generalize the hypothesis

    return h

# Run the FIND-S algorithm
most_specific_hypothesis = find_s_algorithm(concepts, target)

print("Training Data:")
print(df)
print("\nMost Specific Hypothesis (FIND-S):")
print(most_specific_hypothesis)

```

```

... Training Data:
   sky airtemp humidity  wind water forecast enjoysport
0  sunny    warm  normal strong  warm    same         yes
1  sunny    warm   high  strong  warm    same         yes
2  rainy    cold   high  strong  warm  change         no
3  sunny    warm   high  strong  cool  change         yes

```

```

Most Specific Hypothesis (FIND-S):
['sunny', 'warm', '?', 'strong', '?', '?']

```

```

import pandas as pd

# Load the dataset from CSV
df = pd.read_csv('./content/lec3/export.csv')

# Separate concepts and target
concepts = df.iloc[:, :-1].values
target = df.iloc[:, -1].values

def candidate_elimination(concepts, target):

    # Initialize the most specific hypothesis (h)
    specific_h = ['?' for _ in range(len(concepts[0]))]

    # Initialize the general hypothesis (G)
    general_h = [['?' for _ in range(len(concepts[0]))]]

    print("\nInitial Specific Hypothesis:")
    print(specific_h)

    print("\nInitial General Hypothesis:")
    print(general_h)

    for i, h in enumerate(concepts):
        if target[i].lower() == 'yes': # Positive example
            for a in range(len(specific_h)):
                if specific_h[a] == '?':
                    specific_h[a] = h[a]
                elif specific_h[a] != h[a]:
                    specific_h[a] = '?'

            # Remove inconsistent hypotheses from G
            general_h = [g for g in general_h if all(g[j] == '?' or g[j] == specific_h[j] for j in range(len(specific_h)))]

        else: # Negative example
            # Add new inconsistent hypotheses to G
            new_general_h = []
            for g in general_h:
                if all(g[j] == '?' or g[j] != h[j] for j in range(len(specific_h))):
                    # If g is consistent with the negative example, it means it predicts 'No'
                    # But our current setup for specific_h is only for positive examples.
                    # For negative examples, we need to specialize general hypotheses.
                    for j in range(len(specific_h)):
                        if g[j] == '?': # If general hypothesis has '?' for an attribute
                            if specific_h[j] != h[j]: # And specific has value matching negative example
                                temp_g = list(g) # Specialize the general hypothesis
                                temp_g[j] = specific_h[j]
                                new_general_h.append(temp_g)
                            else: # If specific has a different value, then generalize specific
                                pass # No specific generalization from this negative example on this attribute
                        elif g[j] == h[j]: # If general matches the negative example, it is inconsistent, must generalize
                            # This path is more complex for CI, simplifying for now based on usual implementation
                            # The existing generic '?' will handle this. Let's make sure it's valid.
                            pass
                        else: # More robust handling for negative examples (standard Candidate Elimination)
                            for j in range(len(specific_h)):
                                if g[j] == '?' and specific_h[j] != h[j]:
                                    temp_g = list(g)
                                    temp_g[j] = specific_h[j]
                                    if all(temp_g[k] == '?' or temp_g[k] == specific_h[k] for k in range(len(specific_h))):
                                        new_general_h.append(temp_g)
            general_h = [list(x) for x in set(tuple(x) for x in new_general_h)] # Remove duplicates and convert to list of lists
            # Cleanup general_h by removing any hypotheses that are more specific than specific_h (this is implicitly handled by the previous step)

    # Filter out overly general hypotheses that predict 'No' for positive examples already covered by specific_h
    general_h = [g for g in general_h if all(
        all(g_val == '?' or g_val == a_val for g_val, a_val in zip(g, a))
        for a, target_val in zip(concepts, target) if target_val.lower() == 'yes'
    )]

    # Filter out hypotheses that are inconsistent with negative examples
    general_h = [g for g in general_h if not any(
        all(g_val == '?' or g_val == h_val for g_val, h_val in zip(g, h))
        for h, target_val in zip(concepts, target) if target_val.lower() == 'no'
    )]

    return specific_h, general_h

s, g = candidate_elimination(concepts, target)

print("\nFinal Specific Hypothesis:")
print(s)

print("\nFinal General Hypothesis:")

```

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

```

```

# Load Dataset
iris = load_iris()
X = iris.data
y = iris.target

```

```

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

```

```

# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

```

# ANN Model
model = MLPClassifier(hidden_layer_sizes=(10,),
                      max_iter=1000,
                      random_state=42)

```

```

model.fit(X_train, y_train)

```

```

# Prediction
y_pred = model.predict(X_test)

```

```

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

```

```

*** Accuracy: 1.0

Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
  warnings.warn(

```

```
▶ from sklearn.datasets import load_iris
  from sklearn.model_selection import train_test_split
  from sklearn.neighbors import KNeighborsClassifier
  from sklearn.metrics import accuracy_score

  iris = load_iris()

  X = iris.data
  y = iris.target

  X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.3, random_state=42)

  model = KNeighborsClassifier(n_neighbors=3)

  model.fit(X_train, y_train)

  y_pred = model.predict(X_test)

  print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
... Accuracy: 1.0
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

model = GaussianNB()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))

print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

Confusion Matrix

```
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
```

Accuracy: 0.9777777777777777


```

▶ from sklearn.datasets import load_iris
  from sklearn.model_selection import train_test_split
  from sklearn.neighbors import KNeighborsClassifier
  from sklearn.metrics import classification_report

  iris = load_iris()

  X = iris.data
  y = iris.target

  X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.3, random_state=1)

  model = KNeighborsClassifier(n_neighbors=5)

  model.fit(X_train, y_train)

  prediction = model.predict(X_test)

  print(classification_report(y_test, prediction))

```

```

...
              precision    recall  f1-score   support

         0           1.00       1.00       1.00         14
         1           0.95       1.00       0.97         18
         2           1.00       0.92       0.96         13

 accuracy                   0.98         45
 macro avg                  0.98       0.97       0.98         45
 weighted avg              0.98       0.98       0.98         45

```



```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1)

model = GaussianNB()

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print(classification_report(y_test, prediction))
```

...		precision	recall	f1-score	support
	0	1.00	1.00	1.00	14
	1	0.94	0.89	0.91	18
	2	0.86	0.92	0.89	13
	accuracy			0.93	45
	macro avg	0.93	0.94	0.93	45
	weighted avg	0.94	0.93	0.93	45



```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

models = {
    "Logistic Regression": LogisticRegression(max_iter=200),
    "Decision Tree": DecisionTreeClassifier(),
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC()
}

print("Algorithm\t\tAccuracy")

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    acc = accuracy_score(y_test, pred)
    print(f"{name:20} {acc:.2f}")
```

...	Algorithm	Accuracy
	Logistic Regression	1.00
	Decision Tree	1.00
	KNN	1.00
	Naive Bayes	0.98
	SVM	1.00



```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = Perceptron(max_iter=1000)

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, prediction))

*** Accuracy: 0.9333333333333333
```

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn import tree
import matplotlib.pyplot as plt

# Play Tennis Dataset
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain',
               'Overcast', 'Sunny', 'Sunny', 'Rain',
               'Sunny', 'Overcast', 'Overcast', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool',
                   'Cool', 'Mild', 'Cool', 'Mild',
                   'Mild', 'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal',
                'Normal', 'High', 'Normal', 'Normal',
                'Normal', 'High', 'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong',
             'Strong', 'Weak', 'Weak', 'Weak',
             'Strong', 'Strong', 'Weak', 'Strong'],
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No',
                  'Yes', 'No', 'Yes', 'Yes',
                  'Yes', 'No', 'Yes', 'No']
}

df = pd.DataFrame(data)

# Encode categorical values
encoder = LabelEncoder()

for column in df.columns:
    df[column] = encoder.fit_transform(df[column])

X = df.drop('PlayTennis', axis=1)
y = df['PlayTennis']

# Train Decision Tree
model = DecisionTreeClassifier(criterion='entropy')

model.fit(X, y)

# Plot Decision Tree
plt.figure(figsize=(12,7))

tree.plot_tree(model,
               feature_names=['Outlook', 'Temperature', 'Humidity', 'Wind'],
               class_names=['No', 'Yes'],
               filled=True)

plt.title("Decision Tree - Play Tennis Dataset")
plt.show()

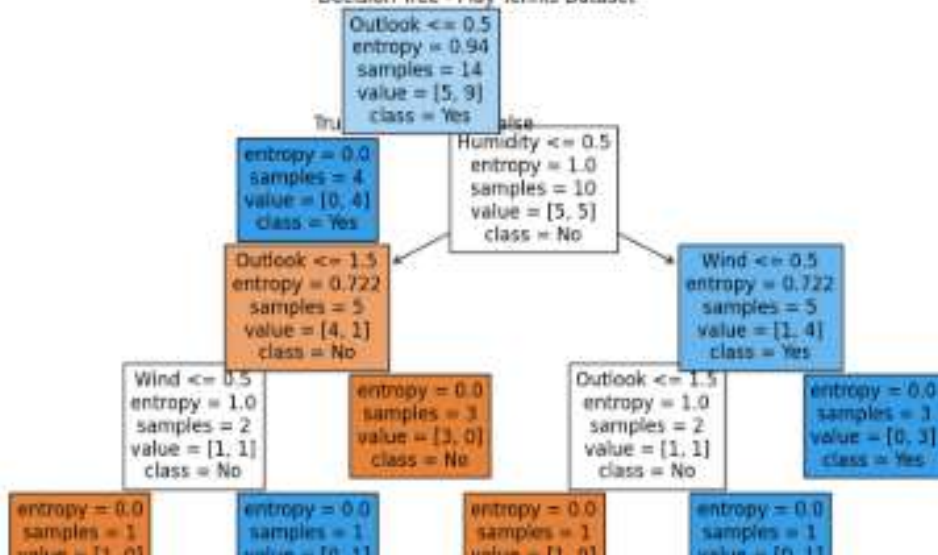
# Prediction
sample = [[2,2,1,1]]

prediction = model.predict(sample)

print("Prediction: ", "Play Tennis" if prediction[0]==1 else "Do Not Play")

```

Decision Tree - Play Tennis Dataset





```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Dataset
data = pd.read_csv("/content/Salary_Data.csv")

X = data[['YearsExperience']]
y = data['Salary']

# Linear Regression
linear = LinearRegression()
linear.fit(X, y)

# Polynomial Regression
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

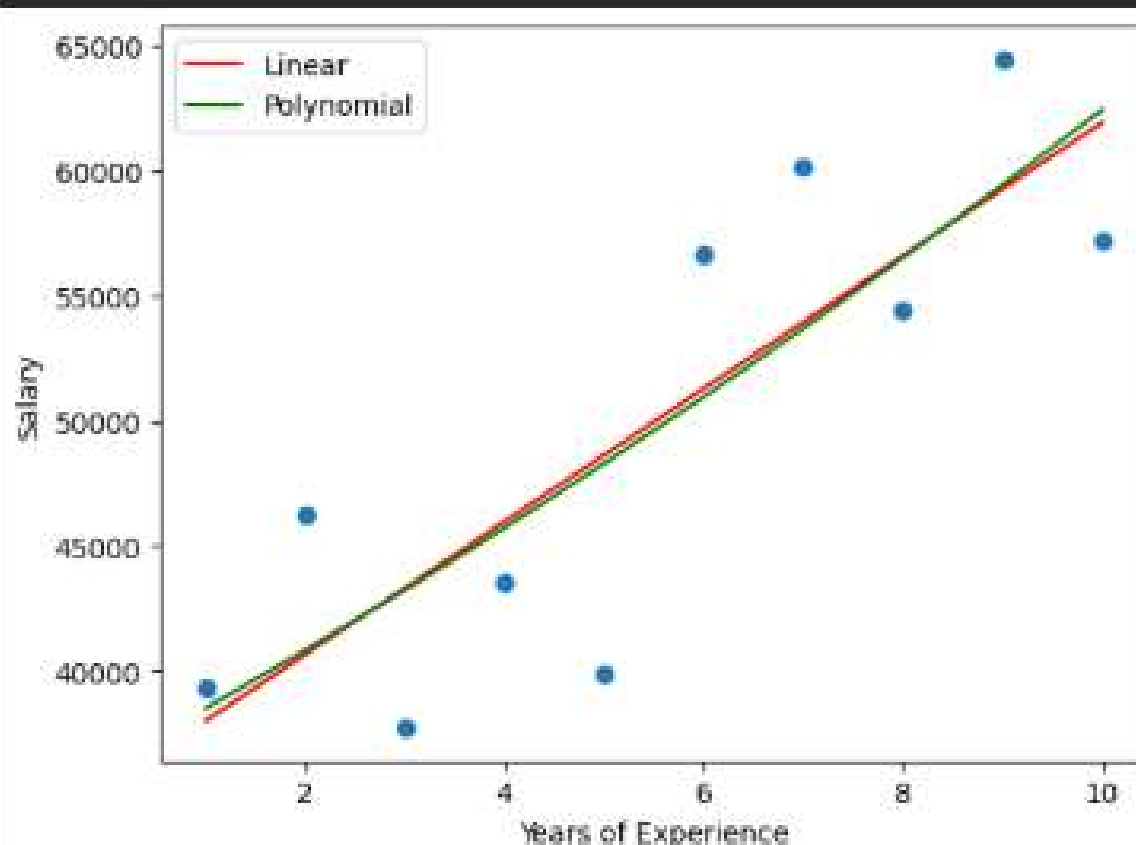
poly_model = LinearRegression()
poly_model.fit(X_poly, y)

# Plot
plt.scatter(X, y)

plt.plot(X, linear.predict(X), color='red', label="Linear")

plt.plot(X, poly_model.predict(X_poly),
         color='green', label="Polynomial")

plt.legend()
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.show()
```



```

import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load Dataset
data = pd.read_csv("/content/Lab 7 DS 1.csv") # Corrected filename

# Rename columns for clarity if it's the 'Play Tennis' dataset and apply label encoding
# Assuming Lab 7 DS 1.csv has columns like 'outlook', 'temp', 'humidity', 'windy', 'play'
# First, let's inspect the data to confirm column names if possible without execution
# If the actual column names are different, this block might need further adjustment.

# Example column names for 'Play Tennis' dataset
data.columns = ['outlook', 'temp', 'humidity', 'windy', 'play']

# Encode categorical values if they are strings
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
for column in data.columns:
    if data[column].dtype == 'object': # Check if column is categorical (object type)
        data[column] = le.fit_transform(data[column])

# Features: Drop the target column 'play'
X = data.drop("play", axis=1)

# Target
y = data["play"]

# Convert categorical columns (already handled by LabelEncoder, get_dummies not strictly needed if all are numeric after encoding)
# X = pd.get_dummies(X) # Not needed after LabelEncoder for this dataset structure

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model
model = RandomForestClassifier()

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Accuracy :", accuracy_score(y_test, prediction))

```

```

... Accuracy : 0.3333333333333333

```

```

import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import LabelEncoder

# Load Dataset
data = pd.read_csv("/content/Lab 7 DS 1.csv") # Corrected filename

# Assuming Lab 7 DS 1.csv has columns for 'Play Tennis' dataset
data.columns = ['outlook', 'temp', 'humidity', 'windy', 'play']

# Encode categorical values
le = LabelEncoder()
for column in data.columns:
    if data[column].dtype == 'object':
        data[column] = le.fit_transform(data[column])

# Features
X = data.drop("play", axis=1)
|
# Target
y = data["play"]

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Model
model = RandomForestRegressor() # Note: RandomForestRegressor for regression, Play Tennis is usually for classification

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Mean Squared Error :", mean_squared_error(y_test, prediction))

```

... Mean Squared Error : 0.17686666666666667



```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

model = KNeighborsClassifier(n_neighbors=3)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
*** Accuracy: 1.0
```



```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

model = GaussianNB()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))

print("\nAccuracy:", accuracy_score(y_test, y_pred))
```

```
*** Confusion Matrix
[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]

Accuracy: 0.9777777777777777
```




```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1)

model = KNeighborsClassifier(n_neighbors=5)

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print(classification_report(y_test, prediction))
```

...		precision	recall	f1-score	support
	0	1.00	1.00	1.00	14
	1	0.95	1.00	0.97	18
	2	1.00	0.92	0.96	13
	accuracy			0.98	45
	macro avg	0.98	0.97	0.98	45
	weighted avg	0.98	0.98	0.98	45



```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

models = {
    "Logistic Regression": LogisticRegression(max_iter=200),
    "Decision Tree": DecisionTreeClassifier(),
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC()
}

print("Algorithm\t\tAccuracy")

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    acc = accuracy_score(y_test, pred)
    print(f"{name:20} {acc:.2f}")
```

```
... Algorithm          Accuracy
Logistic Regression    1.00
Decision Tree          1.00
KNN                    1.00
Naive Bayes            0.98
SVM                    1.00
```



```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = Perceptron(max_iter=1000)

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, prediction))
```

Accuracy: 0.9333333333333333